

`torch.Tensor` is the central data structure in **PyTorch**, a popular deep learning framework. Tensors are similar to NumPy arrays but with additional features that make them suitable for GPU acceleration and automatic differentiation, which are essential for training neural networks.

Key Features of PyTorch Tensors:

- . **Multi-dimensional Arrays:** Tensors can be scalars, vectors, matrices, or higher-dimensional arrays.
- . **GPU Support:** Tensors can be moved to a GPU for faster computation.
- . **Autograd Support:** Tensors can track operations for automatic differentiation.
- . **Rich API:** PyTorch provides a wide range of operations for tensor manipulation.

Creating Tensors

Here are some common ways to create tensors in PyTorch:

1. From Python Lists or NumPy Arrays

```
python
import torch
```

Copy

```
# From a Python list
```

```
tensor_from_list = torch.tensor([1, 2, 3])
```

```
# From a NumPy array
```

```
import numpy as np
numpy_array = np.array([4, 5, 6])
tensor_from_numpy = torch.from_numpy(numpy_array)
```

2. Special Tensor Initialization

```
python
# Create a tensor of zeros
zeros_tensor = torch.zeros(2, 3) # 2x3 tensor filled with zeros
```

Copy

```
# Create a tensor of ones
```

```
ones_tensor = torch.ones(2, 3) # 2x3 tensor filled with ones
```

```
# Create a tensor with random values
```

```
random_tensor = torch.rand(2, 3) # 2x3 tensor with values uniformly sampled from [0, 1)
```

```
# Create a tensor with values sampled from a normal distribution
```

```
normal_tensor = torch.randn(2, 3) # 2x3 tensor with values from a standard normal distribution
```

3. Tensor with a Specific Data Type

```
python
# Specify the data type (e.g., float32, int64)
float_tensor = torch.tensor([1.0, 2.0, 3.0], dtype=torch.float32)
int_tensor = torch.tensor([1, 2, 3], dtype=torch.int64)
```

Copy

Tensor Operations

PyTorch supports a wide range of operations on tensors, such as arithmetic, matrix multiplication, and more.

1. Arithmetic Operations

```
python
```

Copy

```
a = torch.tensor([1, 2, 3])
b = torch.tensor([4, 5, 6])
```

```
# Element-wise addition
c = a + b # tensor([5, 7, 9])
```

```
# Element-wise multiplication
d = a * b # tensor([4, 10, 18])
```

2. Matrix Multiplication

```
python
x = torch.tensor([[1, 2], [3, 4]])
y = torch.tensor([[5, 6], [7, 8]])
```

Copy

```
# Matrix multiplication
z = torch.matmul(x, y) # tensor([[19, 22], [43, 50]])
```

3. Reshaping Tensors

```
python
tensor = torch.arange(6) # tensor([0, 1, 2, 3, 4, 5])
```

Copy

```
# Reshape to 2x3
reshaped_tensor = tensor.view(2, 3) # tensor([[0, 1, 2], [3, 4, 5]])
```

Moving Tensors to GPU

PyTorch allows you to move tensors to a GPU for faster computation.

```
python
# Check if GPU is available
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

Copy

```
# Move tensor to GPU
tensor = torch.tensor([1, 2, 3])
tensor = tensor.to(device)
```

Autograd: Automatic Differentiation

Tensors can track operations for automatic differentiation, which is useful for training neural networks.

```
python
# Enable gradient tracking
x = torch.tensor([2.0, 3.0], requires_grad=True)
```

Copy

```
# Define a computation
y = x[0] * x[1] + x[1]**2 # y = 2*3 + 3^2 = 6 + 9 = 15
```

```
# Compute gradients
y.backward()
```

```
# Access gradients
print(x.grad) # tensor([3., 8.])
```

Common Tensor Attributes

Shape: tensor.shape or tensor.size()

Data Type: tensor.dtype

Device: tensor.device (CPU or GPU)

Gradients: tensor.grad (for autograd)

Example: Full Workflow

Here's an example that combines tensor creation, operations, and autograd:

python

import torch

Create tensors

x = torch.tensor([1.0, 2.0, 3.0], requires_grad=True)

w = torch.tensor([0.5, 0.5, 0.5], requires_grad=True)

b = torch.tensor(1.0, requires_grad=True)

Linear transformation

y = torch.dot(w, x) + b # $y = w \cdot x + b$

Compute gradients

y.backward()

Print gradients

print("Gradient of w:", w.grad) # tensor([1., 2., 3.])

print("Gradient of b:", b.grad) # tensor(1.)

Copy